Relatório Técnico de Projeto: Ecossistema Wiki Zero (WazzimaGiygg Core)

1. Visão Geral e Propósito do Sistema

O ecossistema **WazzimaGiygg Core**, comercialmente identificado como **Wiki Zero**, é uma solução integrada de arquitetura modular para gestão de conhecimento e visualização dinâmica de artigos. Sob a ótica de engenharia de sistemas, o projeto resolve a fragmentação de informações ao centralizar a administração de dados (CRUD) e o consumo de conteúdo numa infraestrutura unificada. A importância estratégica deste repositório reside na sua capacidade de oferecer um portal de acesso público controlado, onde a integridade dos dados é garantida por camadas de auditoria e segurança.

O sistema opera sobre três pilares fundamentais:

- **Autenticação:** Gestão de identidade multinível (Google e Email/Senha).
- **Gestão (CRUD):** Ciclo de vida completo do documento no painel administrativo WZZM.
- **Visualização Dinâmica:** Interface otimizada que resolve parâmetros de URL para entrega de conteúdo.

A natureza modular do sistema é evidenciada pela lógica de `normalizeCollection`. Esta funcionalidade permite que a ferramenta mude instantaneamente o seu contexto operacional — de um repositório pessoal para um sistema académico ou empresarial — bastando alterar os parâmetros `uid` ou `collection` na URL.

2. Arquitetura Tecnológica e Stack de Desenvolvimento

A robustez do Wiki Zero advém de uma arquitetura *serverless* baseada no **Firebase SDK (v8.10.0)**. Esta escolha é crítica para assegurar agilidade no desenvolvimento e escalabilidade automática sem a sobrecarga de gestão de servidores.

Stack Tecnológica Detalhada

Componente	Tecnologia	Observação Técnica
Frontend Base	HTML5, CSS3, JavaScript (ES6+)	Lógica client-side nativa.
Framework UI	Material Design Lite (MDL)	Tema **Indigo-Pink** e Material Icons.
BaaS	Firebase (Firestore & Auth)	Base NoSQL e gestão de tokens de segurança.
Sanitização	DOMPurify (v3.0.6)	Proteção contra ataques XSS em campos de descrição.
Exportação	Html2Canvas	Captura de snapshots da UI para documentação.
Tipografia	Noto Sans / Times New Roman	Noto para interface; Times para preview ABNT.

Nota de Configuração: O sistema utiliza um objeto `firebaseConfig` estático. Embora facilite a portabilidade entre ambientes de teste, em produção, este objeto deve ser revisto sob protocolos de segurança para evitar exposição indevida de chaves de API.

3. Painel Administrativo e Gestão de Conteúdos (WZZM)

O painel **WZZM** é o núcleo operacional onde o ciclo de vida da informação é processado. A interface segue um layout de grade (`grid-template-areas`) com uma barra lateral de listagem e um conteúdo principal dinâmico.

1. **Motor de Busca Avançado:** Suporta filtragem por Título, Descrição, Setor e, crucialmente, por **Document UID**. A busca por UID é um bypass técnico que permite o acesso direto a documentos específicos sem depender da indexação de coleções.
2. **Sistema de Paginação:** Implementado em "estilo Google", limitando o carregamento a **5 itens por página**. Esta lógica reduz o consumo de leitura do Firestore e otimiza o tempo de renderização no DOM.
3. **Campos de Gestão (CRUD):** O sistema impõe uma estrutura de dados rigorosa. No campo "Armário" (Idioma), as opções são restritas a **Português, Inglês e Espanhol**.
4. **Preview ABNT:** O sistema inclui lógica de estilização para visualização de artigos conforme as normas técnicas (ABNT), utilizando fontes *serif* (12pt), alinhamento justificado e recuos de parágrafo de 1.25cm.

4. Sistema de Histórico e Controlo de Versões

O diferencial de integridade do Wiki Zero é o seu sistema de histórico avançado. Cada alteração dispara um snapshot para a coleção `historico`, permitindo auditoria editorial completa.

Diff System e Auditoria: O sistema armazena simultaneamente o estado `versaoAtual` (antes da alteração) e `versaoNova`. Isto permite destacar visualmente as alterações através das classes CSS `diff-added` (verde) e `diff-removed` (vermelho/tachado).

Mecanismo de Reversão: A função `reverterParaVersao` não é apenas uma recuperação de interface; ela executa um `update()` no Firestore utilizando os dados capturados no snapshot histórico, garantindo que o documento original seja restaurado com precisão atómica.

Para fins de auditoria técnica, o sistema monitoriza o `tamanhoCaracteres` vs. `tamanhoAnterior`, além de registar o `editorEmail` e o `timestamp`, assegurando total responsabilidade em ambientes colaborativos.

5. Visualizador de Artigos e Resolução de Parâmetros URL

O Visualizador é uma interface de "consumo limpo" que depende da função `extractArticleParams` para determinar o conteúdo a exibir via parâmetros de consulta (`?uid=...&collection=...`).

Lógica de Regex e URLs Aninhadas: O script possui uma lógica sofisticada de extração via Expressões Regulares (Regex). Se o parâmetro `uid` contiver uma URL completa (ex: de um sistema legado ou aninhado), o sistema extrai automaticamente o `pathUid` ou o `nestedUid`,

permitindo que o visualizador funcione como um *proxy* de conteúdo dentro de outras estruturas de URL.

Coleções Suportadas: O sistema valida a entrada contra um array de `validCollections`, que inclui: `materiadeensaio`, `wikiworldweb`, `materias`, `wikicodigo`, `materia`, `academico`, `userpage`, `historicodecontribuicoes` e a coleção restrita `admin`.

6. Segurança, Autenticação e Perfis de Utilizador

A segurança é multinível, baseada em perfis de acesso: `admin`, `user` e `guest`.

- **Identidade:** O acesso é controlado via Firebase Auth, suportando Google Auth e credenciais tradicionais.
- **Privilégios de Admin:** A interface valida `currentUserRole === 'admin'` para habilitar o `btn-remover-artigo` e conceder acesso à coleção de administração.
- **Telemetria:** O sistema integra um iframe de **Universal Checkuser** (`wazzimagiygg.com/admin/checkuser/universalcheckuser`), que funciona como um mecanismo externo de logging e auditoria de acesso.

7. Conclusão e Notas de Implementação

O WazzimaGiygg Core é uma infraestrutura resiliente para a preservação de conhecimento. Para novos programadores que clonem este repositório, destacam-se os seguintes pontos críticos:

- **Firestore Indexing:** Consultas que combinam filtros (ex: `setor`) com ordenação (`orderBy('ultimaEdicao', 'desc')`) resultarão em erro `failed-precondition`. É obrigatória a criação de índices compostos no console do Firebase conforme indicado nos logs de erro.
- **Sanitização Mandatória:** Todo conteúdo renderizado deve passar por `DOMPurify.sanitize()` para evitar a execução de scripts maliciosos injetados nos campos de descrição.
- **Captura de Ecrã:** A biblioteca `html2canvas` deve ser utilizada para gerar provas visuais do estado dos artigos antes de edições críticas ou remoções.

Este ecossistema está preparado para escala industrial, mantendo a simplicidade de uma Wiki moderna com o rigor de um sistema de gestão documental.